

RECORRIDO DIDÁCTICO

Cómo aprende una IA a crear imágenes

Un viaje visual por los cuatro grandes modelos generativos — de lo más sencillo a lo más avanzado— explicado para que se entienda sin necesidad de saber de inteligencia artificial. Cada tecnicismo, aclarado al momento.

Autoencoder

VAE

GAN

Diffusion

AUTOR

Roberto Canales Mora · con Claude Chat / Code

LICENCIA

MIT

REPOSITORIO

github.com/rcanalescoder/generativeLab

¿De qué va todo esto?

Una escalera de cuatro modelos, del más simple al más capaz

Seguro que has oído hablar de la **inteligencia artificial que crea imágenes**: herramientas que dibujan un retrato, rejuvenecen una foto o inventan un paisaje que no existe. Detrás de casi todas hay un puñado de ideas que se pueden entender sin matemáticas. Este documento las recorre una a una, de la más sencilla a la más potente.

Para hacerlo, construimos un **laboratorio visual**: un pequeño programa donde se entrena cada modelo en directo y se ve —literalmente, con imágenes— qué está aprendiendo. Todo funciona en un portátil corriente, sin servidores ni nube, usando una colección de **43.102 caras de dibujo estilo anime**, cada una de 64×64 píxeles (una imagen diminuta, del tamaño de un icono). El modelo nunca recibe pistas del tipo «esto es pelo» o «esto es un ojo»: solo ve los píxeles y tiene que apañárselas solo.

Los cuatro protagonistas

Cada modelo arregla una carencia del anterior. Esta es la idea de cada uno en una frase; el resto del documento los desarrolla.

Autoencoder	Aprende a resumir una imagen en muy pocos números y luego a reconstruirla a partir de ese resumen. Es la base de todo lo demás.
VAE	El mismo esqueleto, pero ordena su «memoria» de tal forma que ya puede inventar caras nuevas , no solo copiar las que ha visto.
GAN	Pone a competir dos redes —una que falsifica y otra que detecta falsificaciones—. Consigue caras más nítidas, pero entrenarla es delicado.
Diffusion	Aprende a quitar ruido poco a poco , partiendo de una pantalla de «nieve» de tele hasta que aparece una cara. Es la familia detrás de herramientas como DALL·E o Stable Diffusion.

El hilo conductor: mejorar con ayuda de agentes de IA

Hacer que un modelo funcione la primera vez es solo la mitad del trabajo. La otra mitad es **mejorarlo**. Y aquí hicimos algo curioso: en lugar de afinarlo todo a mano, montamos un **bucle de realimentación con agentes de IA** (programas asistentes que proponen cambios, los prueban y comparan resultados). El ciclo es simple: *proponer* una variante → *entrenarla* → *medirla* con cifras objetivas → *quedarse* con la que de verdad mejora. En cada modelo verás el «antes» (la primera versión, con sus defectos) y el «después» (la mejora que el bucle encontró y verificó).

Importante para leer con confianza: **todas las cifras de este documento son reales**, medidas por el propio laboratorio sobre imágenes que el modelo no usó para aprender (su «examen»). Las capturas de pantalla salen directamente de la herramienta. No hay nada inventado ni maquillado.

Autoencoder

Aprender a resumir una imagen en muy pocos números — y reconstruirla

¿Qué es un autoencoder?

Un autoencoder es una red neuronal que aprende a hacer dos cosas seguidas: primero **resumir** una imagen en un puñado de números, y luego **volver a dibujarla** a partir de ese resumen. La parte que resume se llama **encoder** (codificador) y la que vuelve a dibujar, **decoder** (decodificador). Entre las dos hay un «estrechamiento» a propósito: el resumen tiene que caber en muy poco espacio. A ese resumen comprimido se le llama **código latente**, que abreviamos como **z** (es, simplemente, una lista de números que captura «la esencia» de la imagen).



Una analogía

Imagina a un dibujante que mira un retrato durante diez segundos, se va a otra habitación y lo redibuja de memoria. No puede recordar cada píxel, así que se queda con lo importante: «pelo azul largo, ojos grandes, sonríe, fondo claro». Ese puñado de rasgos que apunta mentalmente es el **código latente**; el ejercicio de mirar-y-redibujar de memoria es, exactamente, lo que hace un autoencoder. Y como solo puede recordar unos pocos rasgos, se ve obligado a aprender qué es lo que de verdad importa de una cara.

¿Para qué sirve? (en el mundo real)



Comprimir datos

Guardar imágenes, audio o sensores ocupando mucho menos espacio, quedándose solo con lo esencial.



Quitar ruido

Limpiar fotos granuladas o señales con interferencias: el modelo aprende a reconstruir la versión «limpia».



Detectar anomalías

Si algo es muy raro (un fallo en una máquina, un fraude), el modelo lo reconstruye mal: ese «error grande» es la alarma.

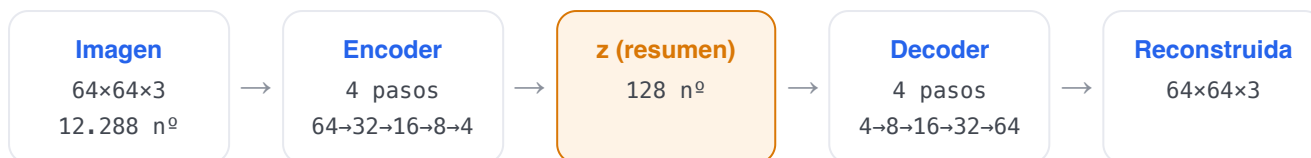


Buscar por parecido

Comparar los resúmenes **z** en vez de los píxeles permite encontrar imágenes parecidas «de fondo», aunque no sean idénticas.

Cómo es la red (su estructura)

La imagen de entrada son 64×64 píxeles en color, y cada píxel tiene 3 valores (rojo, verde, azul): en total **12.288 números**. El encoder los va comprimiendo en cuatro pasos, reduciendo el tamaño a la mitad cada vez (64→32→16→8→4 píxeles de lado) hasta dejar el resumen **z**, que por defecto son solo **128 números**. Es decir, comprime unas **96 veces**. El decoder es el espejo del encoder: parte de esos 128 números y los va ampliando paso a paso (4→8→16→32→64) hasta reconstruir la imagen completa.

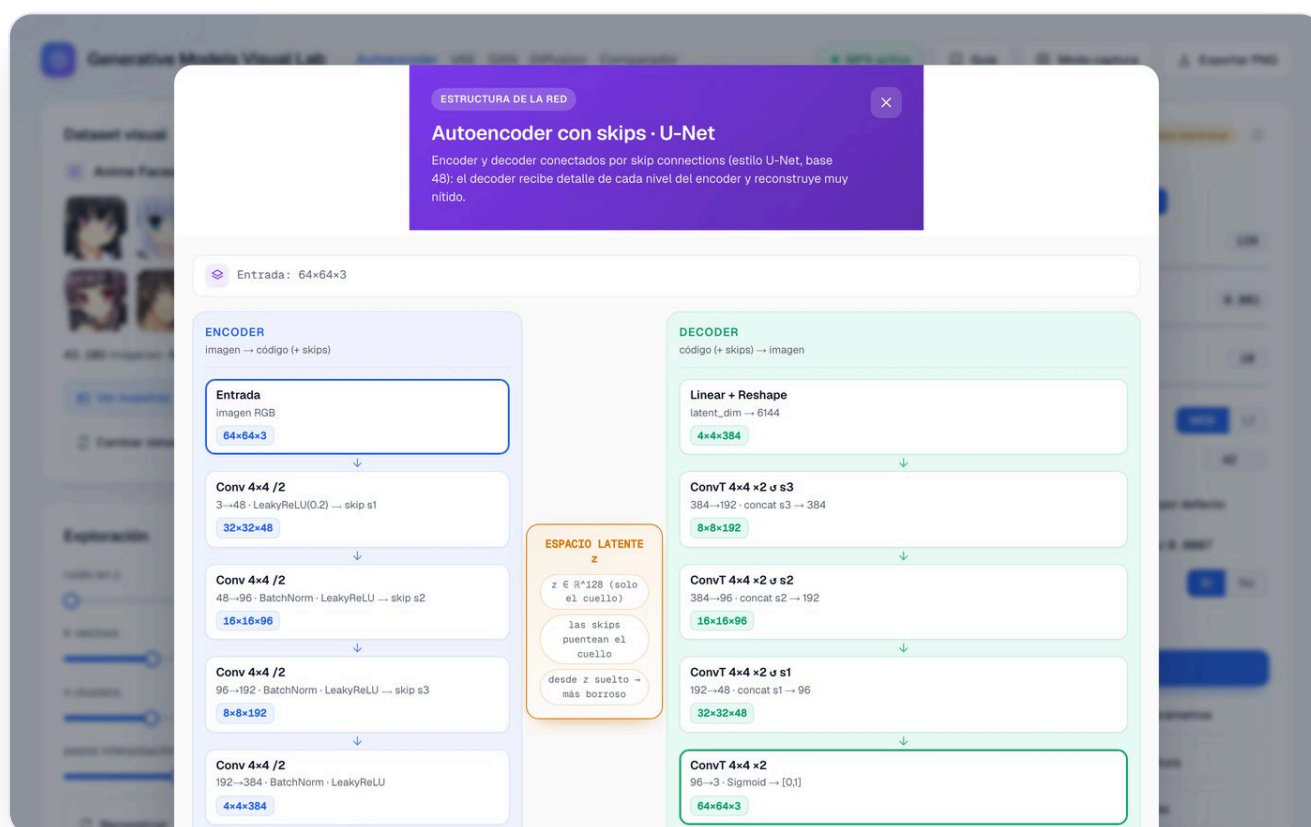


`imagen → z = encoder(imagen) → reconstrucción = decoder(z)`

El truco que lo hace aprender: el «cuello de botella»

Si el resumen z fuera tan grande como la imagen, el modelo haría trampa: copiaría la imagen tal cual sin aprender nada. Al obligarlo a que todo quepa en 128 números, lo forzamos a **descubrir la estructura** de una cara —que tiene ojos, pelo, una pose— y a quedarse solo con eso. Ese estrechamiento se llama *cuello de botella*, y es la idea central de todo el documento.

En la app hay un botón «Ver la estructura» que abre el esquema interactivo capa por capa (encoder a la izquierda, decoder a la derecha y el cuello en medio), como el de la página siguiente.



El esquema «Ver la estructura» de la app (variante U-Net): a la izquierda el encoder comprime la imagen capa a capa; a la derecha el decoder la reconstruye; en el centro, el cuello de botella z . Las flechas que cruzan por arriba son las «conexiones de atajo» que explicamos en los resultados.

El experimento: objetivo, datos y parámetros

Los datos. Entrenamos con el conjunto huggan/anime-faces: **43.102 caras de dibujo (anime) de 64×64 píxeles**, en color y *sin etiquetas* (nadie le dice al modelo qué es cada cosa). **El objetivo:** que la reconstrucción se parezca lo máximo posible al original. El «profesor» es la propia imagen de entrada: el modelo se corrige solo comparando lo que dibuja con lo que vio.

Estos son los ajustes que más influyen en el resultado, explicados sin jerga:

AJUSTE	QUÉ ES Y QUÉ EFECTO TIENE
latent_dim	Cuántos números resumen cada cara (por defecto 128). Pocos números → resumen más abstracto y reconstrucción más borrosa, pero más compresión. Muchos números → más detalle y fidelidad, pero comprime menos.
arquitectura	El «plano» de la red : Básica, Grande o U-Net. Cambia cuánta capacidad tiene y, sobre todo, si usa «conexiones de atajo» (clave en la mejora de más abajo).
loss	Cómo se mide el error entre original y reconstrucción. La opción por defecto (MSE) tiende a suavizar; la alternativa (L1) deja los bordes algo más marcados.
learning_rate	El tamaño de cada paso de aprendizaje . Muy grande: aprende a trompicones y se desestabiliza; muy pequeño: aprende lentísimo.
epochs	Cuántas veces repasa todas las imágenes . Más repasos suelen mejorar, hasta que deja de notarse. Se detiene solo si ya no progresa («parada temprana»).

La interfaz

Generative Models Visual Lab Autoencoder VAE GAN Diffusion Comparador MPS activo Guia Modo captura Exportar PNG

Dataset visual

Anime Faces

43.102 imágenes · 64×64 · RGB · sin etiquetas

Ver muestras Subir imagen Cambiar dataset

Exploración interactivo

ruido en z: 0.00

k vecinos: 8

n clusters: 5

pasos interpolación: 6

Reconstruir Buscar similares Interpolación Calcular clusters

Flujo del modelo

Original 64×64×3 → Encoder (64×64×3 → 4×4×256) → z = 128 → Decoder (4×4×256 → 64×64×3) → Reconstruida 64×64×3

Mapa latente

Proyección 2D de z mediante PCA

Cluster 0, Cluster 1, Cluster 2, Cluster 3, Cluster 4

Cada punto es una imagen codificada por el encoder. Las distancias reales se calculan sobre z, no sobre la proyección 2D.

Entrenamiento requiere reentrenar

arquitectura: Básico, Grande, U-Net

latent_dim: 128

learning_rate: 0.001

epochs: 10

loss: MSE, L1

seed: 42

Volver a parámetros por defecto

modelo entrenado · loss final 0.0159

early stop: Si, No

Rápido, Completo

Entrenar

Buscar mejores parámetros

Ver la estructura

Ver métricas

Resultados del espacio latente

Vecinos similares

Las caras del dataset cuyo código z es más parecido al de la consulta (la primera, en rosa). El número es la distancia: cuanto menor, más parecida según el modelo.

Consulta: 51.17, 52.34, 52.49, 52.66, 53.29, 53.30, 53.69, 53.85

Interpolación A → B

$z_\alpha = (1-\alpha) \cdot z_A + \alpha \cdot z_B$

Tomamos dos caras —A y B— y generamos pasos intermedios mezclando sus códigos. Cada fotograma es una cara nueva decodificada.

A · origen → B · destino

Cambiar B

A, α 0.2, α 0.4, α 0.6, α 0.8, B

α = 0.50 A (0) mezcla B (1)

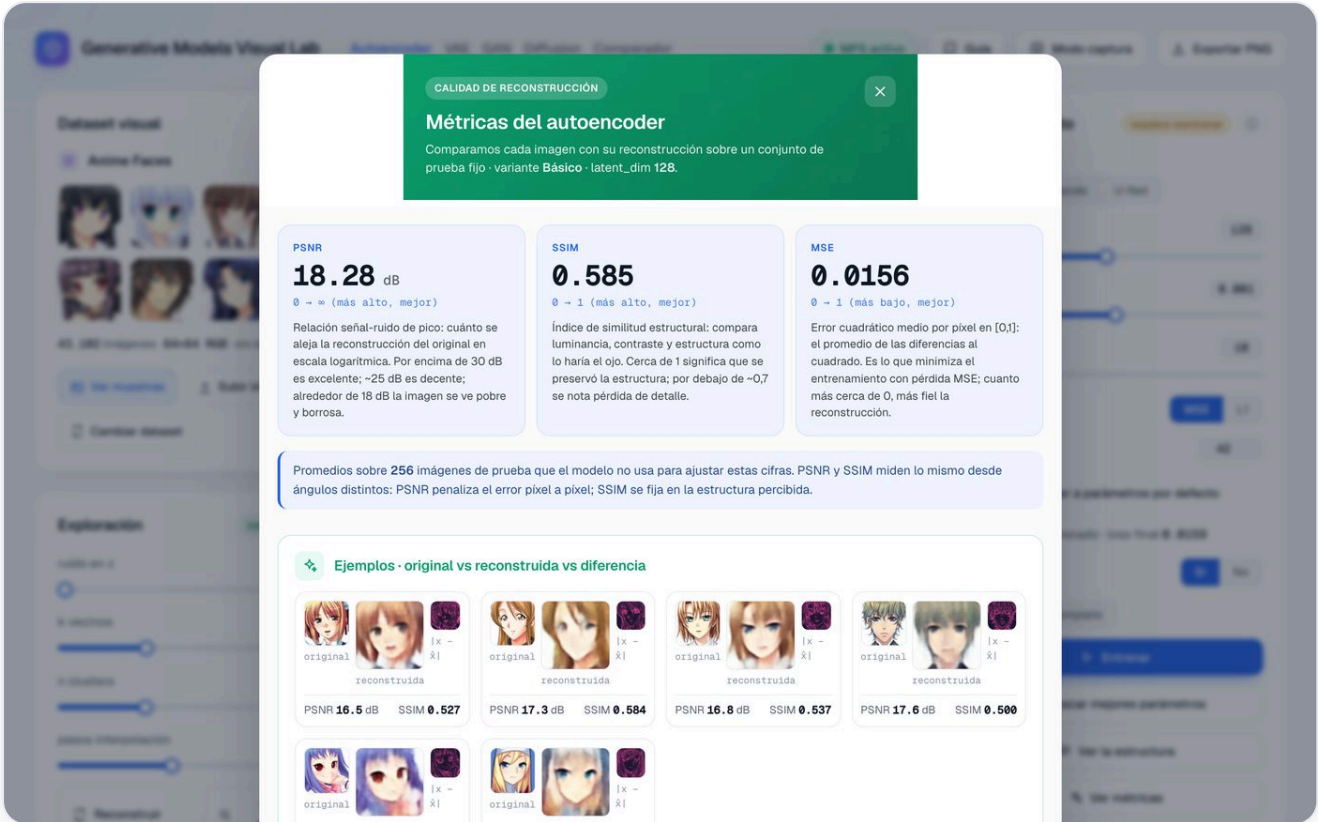
Mueve α para recorrer la transición: en 0 ves A, en 1 ves B, y en medio la mezcla (se resalta el fotograma correspondiente).

La pestaña del Autoencoder en la app. De izquierda a derecha se ve el viaje de una cara: **original** → **encoder** → **z** → **decoder** → **reconstruida** → **mapa de diferencias** (en rosa, dónde se equivoca). Abajo, el «mapa latente» coloca cada cara como un punto según lo que el modelo considera parecido, y a la derecha están los controles para entrenar y experimentar.

Resultados y la mejora con agentes

La primera versión (la **Básica**, con 2,43 millones de parámetros) reconstruye caras perfectamente reconocibles, pero **borrosas**, como una foto desenfocada. No es un fallo de programación: es una consecuencia de cómo mide el error. Cuando el modelo duda entre dos detalles igual de probables (¿el mechón cae a la izquierda o a la derecha?), la opción más «segura» para minimizar el error promedio es

dibujar el punto medio difuso de ambas. A esto lo llamamos cariñosamente la «**maldición del promedio**», y es la razón de que los bordes y las texturas finas salgan suavizados.



Métricas reales de la versión **Básica**, medidas sobre 256 caras de prueba (su «examen»).

⚠ El problema

Caras reconocibles pero claramente **borrosas**. La intuición fácil sería «la red es pequeña, hagámosla más grande». Spoiler: no funcionó.

✓ Cómo se buscó la mejora

Los agentes de IA propusieron, programaron y **midieron** dos variantes: una **Grande** (con más capas y más capacidad) y una **U-Net**, que añade «conexiones de atajo» (en inglés *skip connections*): cables directos que llevan el detalle fino del encoder al decoder, saltándose el cuello de botella.

Comparemos las tres con dos medidas objetivas (las explicamos en las conclusiones; por ahora basta saber que **más alto = más se parece al original**):

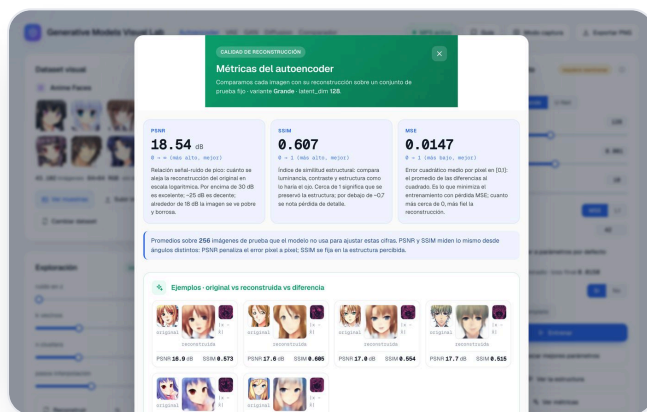


Escala 0–30 dB. Por encima de 30 dB es excelente; unos 18 dB se ve claramente borroso.

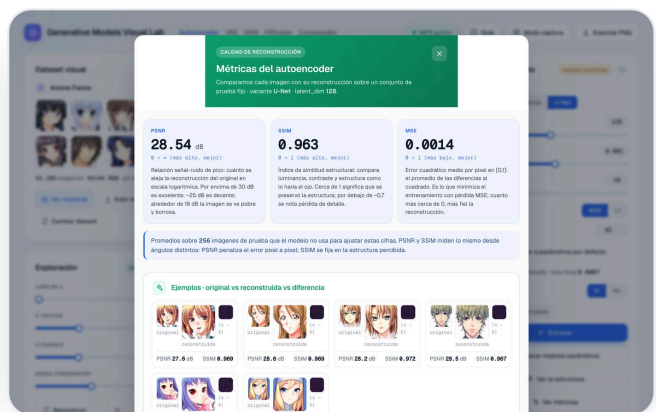
SSIM — parecido «estructural», de 0 a 1 (más alto, mejor)

Básico		0.585
Grande		0.607
U-Net ★		0.963

Variante	Parámetros	PSNR	SSIM	Qué nos dice
Básica	2,43 M	18.28 dB	0.585	El punto de partida: borroso.
Grande	10,72 M	18.54 dB	0.607	4 veces más grande... y casi no mejora. El tamaño no era el problema.
U-Net	5,05 M	28.54 dB	0.963	Las conexiones de atajo lo cambian todo, ¡y con menos piezas que la Grande!



Variante **Grande**: más capacidad, casi la misma borrosidad.



Variante **U-Net**: las conexiones de atajo dan caras nítidas.

Conclusiones

Antes de las cifras, una nota sobre qué significan los dos números que hemos usado:

- **PSNR** mide cuánto se parece la reconstrucción al original, en *decibelios* (la misma unidad del sonido). Por encima de 30 dB es excelente; alrededor de 18 dB se ve borroso. Pasar de 18 a 28 dB es, a ojo, ir de «foto desenfocada» a «foto casi nítida».
- **SSIM** mide el parecido *estructural* (¿están las formas en su sitio?), en una escala de 0 a 1. Pasar de 0,59 a 0,96 significa casi clavar la estructura de la cara.

La lección práctica

Hacer la red **más grande no resolvió nada** (de 18,3 a 18,5 dB pese a cuadruplicar el tamaño): el límite no era la potencia, sino el **diseño**. La mejora de verdad llegó al **cambiar la arquitectura** a U-Net: +10 dB de nitidez y un parecido estructural casi perfecto, además con la mitad de piezas que la variante Grande. Moraleja, válida mucho más allá de la IA: *antes de gastar en más potencia, vale la pena entender dónde está realmente el cuello de botella*.

Un matiz honesto que el laboratorio deja ver: la U-Net reconstruye mejor porque sus atajos «esquivan» el cuello... pero precisamente por eso su resumen *z* guarda menos información. Reconstruir mejor no siempre

significa tener mejor «memoria»: es justo el tipo de compromiso que conviene ver con los propios ojos.

VAE

De copiar a inventar: ordenar la «memoria» para crear caras nuevas

¿Qué es un VAE?

VAE son las siglas de **Variational Autoencoder** (autoencoder «variacional»). Tiene el mismo esqueleto que el autoencoder anterior —un encoder que resume y un decoder que reconstruye—, pero con un cambio decisivo: aprende su «memoria» de forma **ordenada**, de modo que ya no solo sabe copiar caras conocidas, sino **inventar caras nuevas que nunca ha visto**. Es, por tanto, el primer modelo *generativo* de verdad de este recorrido.

¿Cómo lo consigue? En vez de guardar cada cara como un punto exacto, el VAE la guarda como una pequeña «**nube**»: no dice «esta cara está aquí», sino «esta cara está por esta zona, más o menos». Cuando muchas nubes se solapan, el espacio de la memoria queda *relleno*, sin huecos. Y si no hay huecos, puedes señalar cualquier punto al azar, pedirle al decoder que lo dibuje, y saldrá una cara plausible.



Una analogía

El autoencoder ordena sus recuerdos como **chinchetas sueltas** en un corcho: entre chincheta y chincheta hay zonas vacías, y si pinchas en el vacío no hay nada. El VAE, en cambio, ordena sus recuerdos como **ciudades en un mapa de población**: cada cara es una mancha difusa, las manchas vecinas se tocan y el territorio queda cubierto. Por eso puedes plantar el dedo en cualquier punto del mapa y siempre caes en «zona habitada»: una cara con sentido.

¿Para qué sirve? (en el mundo real)



Generar variaciones

Crear muchas versiones distintas de algo (caras, productos, moléculas) explorando su «mapa» de posibilidades.



Datos sintéticos

Fabricar ejemplos artificiales realistas para entrenar otros sistemas cuando los datos reales escasean o son sensibles.



Editar con «mandos»

Como la memoria está ordenada, a veces se encuentran ejes que controlan rasgos (sonrisa, edad) y permiten retocarlos a voluntad.



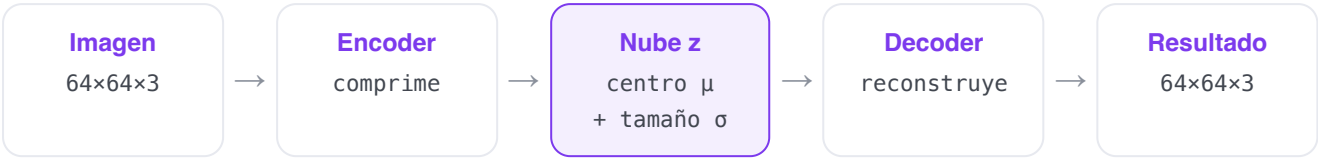
Detección de anomalías

Igual que el autoencoder, pero con una medida de «cuán probable» es algo, útil en medicina o control de calidad.

¿Cómo es la red (su estructura)

La columna vertebral es idéntica a la del autoencoder (encoder de 4 pasos que comprime $64 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 4$, y decoder simétrico que amplía de vuelta). La única diferencia está en el cuello de botella: en lugar de producir un único resumen, el encoder produce **dos**: el **centro de la nube** (su

posición, que llamamos μ) y su **tamaño** (cuánta incertidumbre tiene, que llamamos σ). Para reconstruir o generar, se toma un punto al azar dentro de esa nube y se le pasa al decoder. En total son unos **3,0 millones de parámetros**.



La pieza que ordena la memoria

Durante el entrenamiento, el VAE añade una «fuerza de orden» (su nombre técnico es **término KL**) que empuja todas las nubes hacia un mismo molde común y centrado. Eso impide que el modelo haga trampa encogiendo las nubes a puntos —que sería volver al autoencoder— y es lo que deja el mapa relleno y sin huecos. Sin esa fuerza, no se podría generar.

El experimento: objetivo, datos y parámetros

Mismos **datos** que antes (las 43.102 caras de anime de 64x64, sin etiquetas) y mismo método de entrenamiento. La diferencia está en el **objetivo**: además de reconstruir bien, el modelo debe mantener su memoria ordenada. Por eso su «nota» combina dos cosas: cuánto se parece la reconstrucción al original y cuán ordenadas están las nubes. El ajuste estrella aquí es la letra griega β (beta), que decide a cuál de las dos dar más importancia.

AJUSTE	QUÉ ES Y QUÉ EFECTO TIENE
β (beta)	El equilibrio clave del VAE, exclusivo de este modelo. Decide cuánto importa tener la memoria ordenada frente a reconstruir con fidelidad. β bajo $\rightarrow$ caras más nítidas pero memoria con huecos (peor para inventar); β alto $\rightarrow$ memoria muy ordenada pero caras más borrosas. El valor por defecto (1) es el equilibrio clásico.
latent_dim	Cuántos números resumen cada cara, igual que en el autoencoder (128 por defecto).
loss · learning_rate · epochs	Cómo se mide el error, el tamaño del paso de aprendizaje y cuántos repazos se dan, exactamente igual que en el autoencoder.

La interfaz

Generative Models Visual Lab

Autoencoder VAE GAN Diffusion Comparador

MPS activo

Guía

Modo captura

Exportar PNG

Dataset visual

Anime Faces

43.102 imágenes · 64×64 RGB · sin etiquetas

Ver muestras

Subir imagen

Cambiar dataset

Exploración

interactivo

ruido en z: 0.00

k vecinos: 8

n clusters: 5

pasos interpolación: 6

Reconstruir

Buscar similares

Interpolación

Calcular clusters

Flujo del modelo

Mapa latente

Proyección 2D de z mediante PCA

Cada punto es una imagen codificada por el encoder. Las distancias reales se calculan sobre z, no sobre la proyección 2D.

Generación (muestrear del prior)

$z \sim N(0, I)$

Tomamos vectores z al azar del prior $N(0, I)$ y los pasamos por el decoder. No hay imagen de entrada: cada cara es nueva. Es lo que un autoencoder normal no puede hacer.

nº de muestras: 8

semilla: 42

Generar nuevas

Entrenamiento

requiere reentrenar

latent_dim: 128

learning_rate: 0.001

epochs: 12

loss: MSE L1

β (peso del KL): 1.00

seed: 42

Volver a parámetros por defecto

modelo entrenado · loss final 0.1011

early stop: Sí No

Rápido Completo

Entrenar

Buscar mejores parámetros

Ver la estructura

Resultados del espacio latente

Vecinos similares

Las caras del dataset cuyo código z es más parecido al de la consulta (la primera, en rosa). El número es la distancia: cuanto menor, más parecida según el modelo.

Interpolación A → B

$z_\alpha = (1-\alpha) \cdot z_A + \alpha \cdot z_B$

Tomamos dos caras —A y B— y generamos pasos intermedios mezclando sus códigos. Cada fotograma es una cara nueva decodificada.

$\alpha = 0.50$

A (0) mezcla B (1)

Mueve α para recorrer la transición: en 0 ves A, en 1 ves B, y en medio la mezcla (se resalta el fotograma correspondiente).

La pestaña del VAE. Reconoces el flujo y el mapa latente del autoencoder, pero ahora hay algo nuevo y especial: un panel de **Generación** que crea caras desde cero —sin partir de ninguna foto— simplemente señalando puntos al azar en el mapa de la memoria y pidiéndole al decoder que los dibuje.

Resultados y la mejora con agentes

El VAE consigue lo que el autoencoder no podía: **inventar caras nuevas**. A cambio, las caras que genera salen algo **suaves o «promediadas»**. Y aquí la lección es interesante, porque el problema *no* tiene una solución dentro del propio VAE: el desenfoque viene de dos sitios a la vez —de medir el error píxel a píxel (la misma «maldición del promedio» del autoencoder) y de la propia fuerza de orden, que al solapar nubes empuja al decoder a dibujar algo «medio» que valga para todas.

⚠ El problema

Ya genera caras nuevas, pero **borrosas**. Y no es un bug que se pueda parchear: el desenfoque es *intrínseco* a cómo funciona el modelo.

✓ Qué aportó el bucle

Dos cosas. Una práctica: reutilizar la columna convolucional mejorada del autoencoder y dejar **β como un mando interactivo** para que cualquiera explore el compromiso en vivo. Y una conceptual, más valiosa: entender que para nitidez de verdad hace falta **cambiar de enfoque**, no afinar este. Justo lo que harán los dos modelos siguientes.

El comportamiento de β resume toda la «personalidad» del VAE en una tabla:

Valor de β	Efecto en la reconstrucción	Efecto en la capacidad de inventar
$\beta = 0$	Más nítida (se comporta como un autoencoder)	La memoria vuelve a tener huecos: señalar al azar da imágenes rotas.
$\beta = 1$	Algo más borrosa	El equilibrio: se puede generar e interpolar con sentido.
$\beta > 1$	Más borrosa	Memoria muy ordenada y suave; a veces aparecen «mandos» de rasgos.

Conclusiones

La lección práctica

El VAE da el salto de **copiar a inventar** con un truco elegante: guardar cada recuerdo como una nube difusa para que el «mapa» de la memoria quede relleno. El precio es cierta borrosidad, y la enseñanza más útil es saber **distinguir un defecto que se arregla de una limitación de fondo**: aquí el desenfoque es de fondo, y reconocerlo nos dice exactamente por qué necesitamos los dos modelos siguientes. Además, exponer el mando β ilustra una idea general de la IA: casi todo es un *compromiso*, y verlo con un control deslizante vale más que mil fórmulas.

GAN

Un falsificador contra un detective: la competición que da nitidez

¿Qué es una GAN?

GAN son las siglas de **Generative Adversarial Network** (red generativa «adversaria», es decir, basada en una rivalidad). La idea es radicalmente distinta a las anteriores: aquí no hay un encoder que resuma imágenes. En su lugar, **se enfrentan dos redes que aprenden compitiendo**. Una, el **generador**, fabrica caras partiendo de números al azar. La otra, el **discriminador**, hace de juez: recibe una cara y decide si es real (del conjunto de datos) o falsa (recién fabricada por el generador).

El generador quiere engañar al juez; el juez quiere no dejarse engañar. Cada vez que uno mejora, obliga al otro a mejorar también, y así van subiendo el nivel los dos a la vez. La clave: como el «control de calidad» lo hace *otra red* (y no una simple comparación píxel a píxel), desaparece la «maldición del promedio» y las caras salen **mucho más nítidas**.



Una analogía

Un **falsificador de billetes** (el generador) y un **detective** (el discriminador) que aprenden a la vez. Al principio el falsificador hace chapuzas y el detective las pilla sin esfuerzo. Pero con cada billete descubierto, el falsificador refina su técnica; y con cada billete que se le cuele, el detective agudiza el ojo. Tras miles de rondas, los billetes falsos son tan buenos que ni el experto los distingue. Ese «experto cada vez más exigente» es lo que empuja al generador a la excelencia.

¿Para qué sirve? (en el mundo real)



Superresolución

Convertir imágenes de baja calidad en versiones nítidas y detalladas (fotos antiguas, imágenes médicas, satélite).



Avatares y «deepfakes»

Generar rostros y personas que no existen, para videojuegos, cine o avatares; también la tecnología tras los polémicos deepfakes.



Arte y diseño

Crear texturas, estilos y obras visuales; transformar fotos en pinturas o cambiar el estilo de una imagen.



Moda y producto

Diseñar prendas, muebles o envases ficticios pero realistas para inspirar o prototipar antes de fabricar.

¿Cómo es la red (su estructura)

Son dos redes espejo. El **generador** hace el camino del decoder, pero al revés de empezar por una imagen: arranca de un vector de **100 números al azar** (el «germen» de cada cara) y los va ampliando hasta una imagen de 64×64, pasando por capas cada vez más anchas (512→256→128→64 canales). El **discriminador** hace el camino contrario: recibe una imagen de 64×64 y la va reduciendo hasta escupir

un **único número**, su veredicto «real o falsa». Juntos suman unos **6,3 millones de parámetros** (3,6 el generador y 2,8 el discriminador). Es la receta clásica conocida como **DCGAN**.



azar $z \rightarrow G(z) = \text{cara falsa} \rightarrow D(\text{cara}) = \text{¿real o falsa?}$

El camino va en un solo sentido

A diferencia del autoencoder y el VAE, la GAN **no tiene encoder**: solo sabe ir de «números al azar» a «cara», nunca al revés. Por eso aquí no se puede «reconstruir esta foto concreta»; lo que sí se puede es pasear por el espacio de los números de entrada y ver cómo va cambiando la cara que produce.

El experimento: objetivo, datos y parámetros

Los mismos **datos** (43.102 caras de anime de 64×64, sin etiquetas). El **objetivo** ya no es reconstruir, sino que el generador aprenda a fabricar caras tan convincentes que el juez no las distinga de las reales. Es un entrenamiento peculiar: en vez de una sola red bajando una «nota», son dos redes tirando una de la otra.

AJUSTE	QUÉ ES Y QUÉ EFECTO TIENE
z_dim	Cuántos números al azar alimentan al generador (por defecto 100). Es la «semilla» de cada cara: más números dan más variedad posible, pero son más difíciles de cubrir bien.
learning_rate	El tamaño del paso de aprendizaje , y aquí es especialmente delicado: si es muy grande, la competición se desequilibra y se estropea. El valor por defecto (0,0002) es el clásico que funciona.
epochs	Cuántas rondas dura el duelo . Más rondas suelen mejorar la calidad, hasta que el juego se estanca o se desestabiliza.
batch_size	Cuántas imágenes se ven de golpe en cada paso (128 por defecto). Lotes grandes estabilizan algo el entrenamiento, a cambio de más memoria.

La interfaz

Generative Models Visual Lab Autoencoder VAE **GAN** Diffusion Comparador MPS activo Guía Modo captura Exportar PNG

Galería generada

Cada cara nace de un vector de ruido $z \sim N(0,1)$ que el generador convierte en imagen. Ninguna existe en el dataset.

Interactivo ⓘ

Entrenamiento requiere reentrenar ⓘ

z_dim 100

learning_rate 0.0002

epochs 25

seed 42

Volver a parámetros por defecto

modelo entrenado · g_loss 5.5180 · d_loss 0.3794

Entrena para ver las curvas de pérdida de G y D en vivo.

Rápido Completo

▶ Entrenar

⌵ Ver la estructura

n° de muestras 16 seed 42

⚡ Generar ↻ Generar nuevas

Interpolación en z

Caminamos por el espacio de ruido entre dos puntos aleatorios. Como la GAN no tiene encoder, los extremos son ruido, no caras del dataset.

Interactivo ⓘ

$z_\alpha = (1-\alpha) \cdot z_A + \alpha \cdot z_B$

A 0.143 0.286 0.429 0.571 0.714 0.857 B

pasos 8 seed 7

↗ Interpoliar ↻ Nueva interpolación

La pestaña de la GAN. Arriba, una galería de caras **generadas desde cero** (ninguna existe en el conjunto de datos). En medio, un paseo que mezcla dos «semillas» al azar y muestra la transición. Abajo, las dos curvas del duelo: el error del generador y el del juez, que se persiguen mutuamente.

Resultados y la mejora con agentes

La GAN cumple su promesa —caras notablemente más nítidas que las del VAE— pero su entrenamiento es famoso por ser **inestable**. A diferencia de los otros modelos, aquí no hay una única «nota» que baje tranquilamente: hay un equilibrio móvil entre dos rivales, y mantenerlo es delicado. Dos cosas pueden

torcerse: que el juez se vuelva *demasiado* bueno y aplaste al generador (que entonces deja de recibir pistas útiles y se atasca), o que el generador encuentre un atajo —unas pocas caras que siempre cuelan— y se limite a repetirlas, perdiendo variedad. A esto último se le llama **colapso de modo**: la galería se llena de caras casi idénticas.

⚠ El problema

El duelo se desequilibra con facilidad: o el juez «gana» y el generador deja de aprender, o aparece el **colapso de modo** (todas las caras se parecen). Sin cuidado, no sale nada decente.

✓ La mejora

El bucle aplicó las **recetas de estabilización ya verificadas** de la literatura (la familia DCGAN): ciertos tipos de capa y de activación, un optimizador con ajustes que amortiguan las oscilaciones del duelo, una forma de medir el error que da pistas más útiles cuando el generador aún es malo, y normalizar los datos a un rango concreto. Con todo ello el juego se equilibra y produce caras reconocibles.

La GAN suma unos **6,3 millones de parámetros**. La calidad es modesta a propósito: es un modelo pequeño entrenado pocas rondas en un portátil. El objetivo aquí no es competir con los sistemas gigantes, sino **ver el mecanismo** de la rivalidad funcionando con tus propios ojos.

I Conclusiones

La lección práctica

La GAN demuestra que **cambiar quién juzga la calidad lo cambia todo**: poner a otra red de «control de calidad», en vez de una comparación mecánica, rompe la borrosidad y da nitidez. Pero ese poder viene con un coste real —un entrenamiento **delicado e inestable**— y con conceptos propios como el colapso de modo. La enseñanza general: en IA, las técnicas más potentes suelen ser también las más temperamentales, y a menudo lo decisivo no es inventar de cero, sino **aplicar bien las recetas que ya se sabe que funcionan**.

Diffusion

De una pantalla de «nieve» a una cara, quitando ruido poco a poco

¿Qué es un modelo de difusión?

Es la familia de modelos que hoy domina la generación de imágenes: detrás de herramientas como **Stable Diffusion**, **DALL·E** o **Midjourney** hay esta misma idea. Y la idea es sorprendentemente sencilla: **aprender a quitar ruido**. El modelo se entrena para que, dada una imagen ensuciada con un poco de ruido (esa «nieve» de las teles antiguas sin señal), adivine ese ruido y lo reste. ¿Y para generar una cara desde cero? Se parte de una pantalla de ruido *puro* y se aplica esa operación muchas veces seguidas: quita un poco de ruido, otro poco, otro poco... hasta que, paso a paso, emerge una cara.

Lo elegante del truco es que convierte **un problema imposible en muchos problemas fáciles**. Pedirle a una red «dibuja una cara de la nada» sería difícilísimo; pero «a esta imagen casi limpia, quítale el poquito de ruido que le queda» es una tarea sencilla y estable de aprender. Encadenando muchas tareas fáciles se logra lo que de un golpe sería imposible.



Una analogía

Como un **escultor ante un bloque de mármol**. No saca la figura de un solo martillazo: da muchos golpecitos pequeños, retirando un poco de material cada vez, y la figura va «apareciendo». La difusión hace lo mismo, pero retirando ruido en lugar de piedra: empieza con un bloque de pura «nieve» y, golpecito a golpecito, deja al descubierto la cara que había escondida dentro del azar.

¿Para qué sirve? (en el mundo real)



Crear imágenes desde texto

Es el motor de Stable Diffusion, DALL·E o Midjourney: describes algo con palabras y te lo dibuja.



Rellenar y reparar (inpainting)

Borrar un objeto de una foto y reconstruir el fondo, o restaurar zonas dañadas de forma convincente.



Ampliar y mejorar

Aumentar la resolución de imágenes y añadir detalle realista, igual que la superresolución de las GAN.



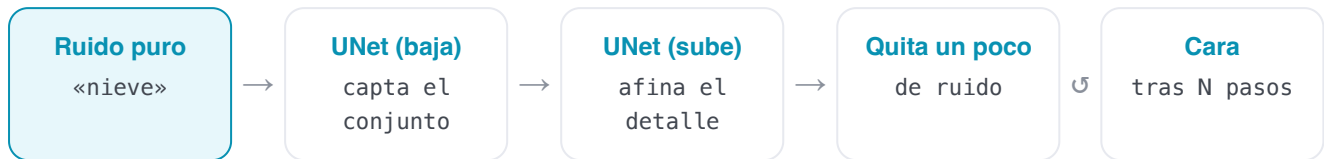
Vídeo, audio y 3D

La misma idea de «quitar ruido paso a paso» se usa ya para generar música, voz, vídeo e incluso modelos en 3D.

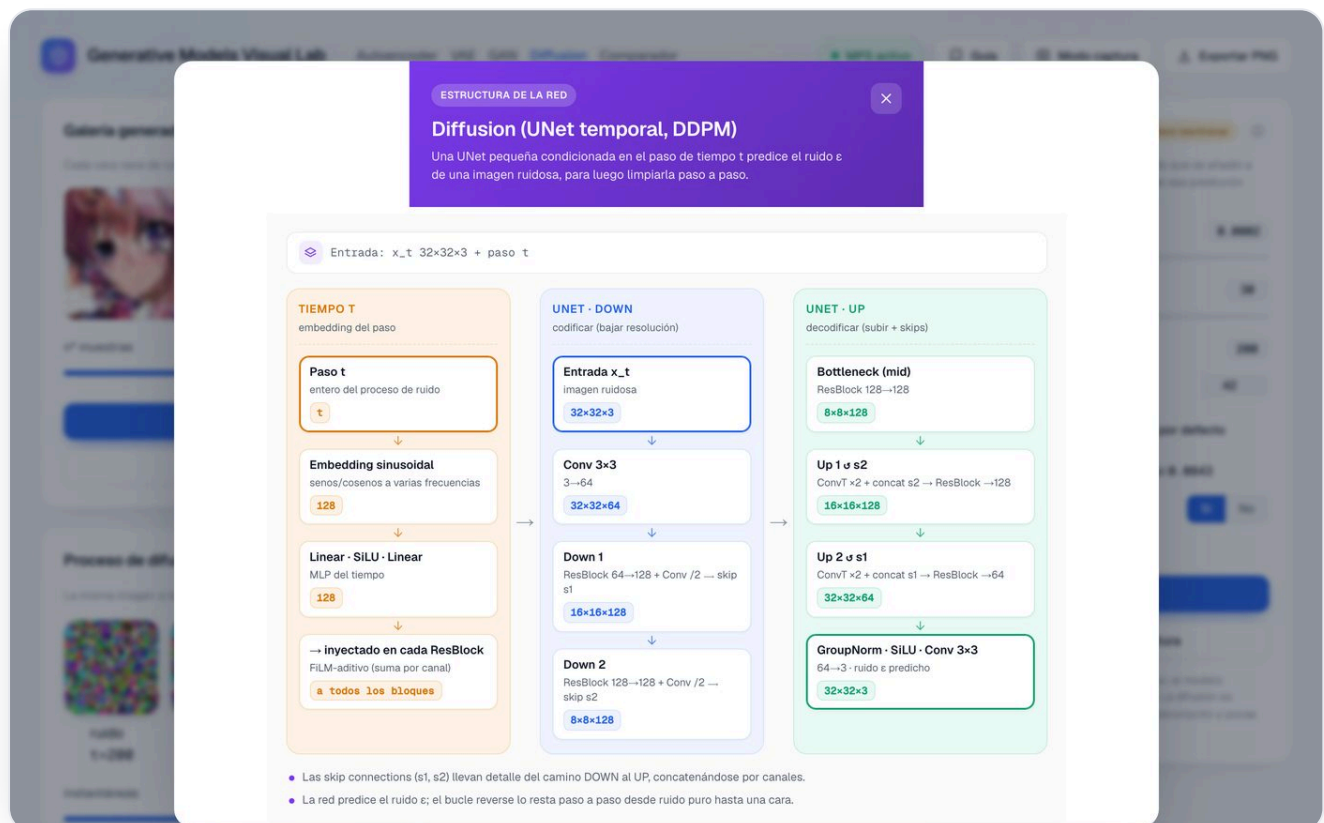
¿Cómo es la red (su estructura)

El corazón es una red con forma de **U** (de ahí su nombre, **UNet**): primero reduce la imagen para entender el «conjunto» y luego la vuelve a ampliar para afinar el detalle, con cables de atajo que conectan ambos lados. Lo distintivo es que, además de la imagen ruidosa, recibe un dato extra: **cuánto ruido tiene** en ese momento (un número que llamamos *t*, el «paso» del proceso), para saber con cuánta «nieve» está

lidiando. La versión de calidad de esta app trabaja directamente a **64×64 píxeles** e incluye un mecanismo llamado **atención**, que permite a cada zona de la imagen «mirar» a todas las demás para mantener la coherencia global (que los dos ojos combinen, que el peinado sea simétrico). Esa versión nítida tiene unos **21 millones de parámetros**.



En la app, el botón «Ver la estructura» abre el esquema de la UNet (a la izquierda baja la resolución, a la derecha la sube, con atajos cruzados y la inyección del «reloj» t en cada bloque), como el de abajo.



El esquema «Ver la estructura» de la UNet en la app (mostrando la variante ágil): a la izquierda la columna del «reloj» t que indica cuánto ruido hay; en el centro, la red bajando la resolución; a la derecha, subiéndola de nuevo con atajos para reconstruir. La red no dibuja la cara: predice el ruido que sobra, y el bucle lo va restando.

El experimento: objetivo, datos y parámetros

Los mismos **datos** (43.102 caras de anime de 64×64, sin etiquetas). El **objetivo de entrenamiento** es de lo más simple: se coge una cara real, se le añade una cantidad de ruido elegida al azar, y se le pide a la red que adivine *exactamente* el ruido que se añadió. Nada más. Como ese único «adivina el ruido» vale para cualquier nivel de suciedad, luego se puede encadenar muchas veces para generar.

AJUSTE

QUÉ ES Y QUÉ EFECTO TIENE

pasos de muestreo

Cuántos «golpecitos» se dan al generar una cara. Más pasos → transición más fina y mejor calidad, pero más lento. Es el compromiso central de la difusión: calidad contra tiempo.

timesteps (T)

La longitud total del proceso de ruido con el que se entrena (200 por defecto). Es el «mapa completo»; los pasos de muestreo deciden cuántas paradas hacemos al recorrerlo.

schedule

El ritmo al que se añade el ruido. El que usamos por defecto («cosine») destruye el detalle más despacio, lo que a este tamaño pequeño mejora bastante el resultado.

learning_rate · epochs

El tamaño del paso de aprendizaje y cuántos repases se dan, igual que en los demás modelos.

La interfaz

The screenshot shows the 'Generative Models Visual Lab' interface with the 'Diffusion' tab selected. The interface is divided into several sections:

- Top Bar:** Includes navigation links (Autoencoder, VAE, GAN, Diffusion, Comparador), a status indicator 'MPS activo', and buttons for 'Guía', 'Modo captura', and 'Exportar PNG'.
- Galería generada:** Displays a row of six generated anime-style faces. Below them are sliders for 'n° muestras' (set to 6), 'pasos de muestreo' (set to 50), and 'seed' (set to 42). Buttons for 'Generar nuevas' and 'Nueva semilla' are present.
- Proceso de difusión:** A horizontal sequence of eight images showing the progression from pure noise to a clear face. The images are labeled with time steps: 'ruido t=200', 't 175 7/50', 't 146 14/50', 't 118 21/50', 't 85 29/50', 't 57 36/50', 't 28 43/50', and 'cara t=0'. Below the images are sliders for 'Instantáneas' (set to 8), 'pasos de muestreo' (set to 50), and 'seed' (set to 7). Buttons for 'Nuevo proceso' and 'Nueva semilla' are at the bottom.
- Entrenamiento:** Contains training parameters and controls. It includes a 'requiere reentrenar' warning, a description of the model's task, and sliders for 'arquitectura' (Agil/Nitido), 'learning_rate' (set to 0.0002), 'epochs' (set to 60), 'timesteps (T)' (set to 200), and 'seed' (set to 42). A 'Volver a parámetros por defecto' button is also present. The training status shows 'modelo entrenado - loss final 0.0621'. There are buttons for 'early stop' (Si/No), 'Rápido' (selected), and 'Completo'. A large blue 'Entrenar' button is at the bottom, along with a 'Ver la estructura' button.

La pestaña de Diffusion. Arriba, la galería de caras generadas. Pero la tarjeta estrella es el «**Proceso de difusión**»: una tira de fotogramas que muestra, literalmente, el viaje de **ruido puro (izquierda)** → **cara limpia (derecha)**, paso a paso. Es la mejor forma de ver cómo el modelo «esculpe» la cara a partir del azar.

Resultados y la mejora con agentes

Este fue el caso más instructivo de todos, porque el primer intento **generaba manchas de colores, no caras** —incluso al final del proceso—. Lo desconcertante era que el entrenamiento iba *bien*: el error de la red bajaba con normalidad (de 0,50 a 0,084), señal inequívoca de que la red **sí** estaba aprendiendo a detectar ruido. ¿Cómo podía aprender bien y aun así generar basura?

La depuración con agentes destapó un detalle revelador: **el modelo estaba perfecto; el fallo estaba en cómo se le pedía generar**. El procedimiento original quitaba el ruido en pasos que solo sabían avanzar «de uno en uno», pero para ir rápido se saltaba la mayoría: daba 50 pasitos cuando el ruido requería 200. Resultado: solo retiraba una cuarta parte de la «nieve» y la salida seguía siendo, literalmente, ruido.

⚠ La causa (depuración con agentes)

El método de generación solo sabía avanzar paso a paso, pero se saltaba la mayoría (50 de 200) para ir rápido. Así nunca terminaba de limpiar la imagen. **El modelo no tenía la culpa: el procedimiento de generado estaba roto.**

✓ La solución

Se cambió a un método de generación moderno (**DDIM**) que sí sabe *dar zancadas largas y limpiar de golpe* sin perder el rumbo. Se añadió promediar los pesos del modelo para más estabilidad y el ritmo de ruido «cosine». Y un detalle precioso: las primeras caras reconocibles aparecieron **sin reentrenar nada**, porque el problema nunca estuvo en el aprendizaje.

Con la generación arreglada, el siguiente paso del bucle fue subir la calidad: se pasó de una red pequeña que trabajaba en miniatura (32×32) a una **red mayor que trabaja directamente a 64×64 y usa atención** (los 21 millones de parámetros). El resultado son caras claramente reconocibles:



Caras generadas por el modelo de difusión **nítido** (a 64×64, con atención) tras arreglar la generación y ampliar la red. Cada una ha nacido de una pantalla de ruido puro, esculpida paso a paso. Ninguna existe en el conjunto de datos.

Aspecto	Antes (no funcionaba)	Después (funciona)
Cómo se genera	Pasos «de uno en uno», pero saltándose casi todos	DDIM: da zancadas largas y limpia bien
Pesos al generar	Los del último momento (inestables)	Un promedio suavizado (más estables)
Tamaño de trabajo	Miniatura 32x32, red pequeña	64x64 nativo, red mayor con atención (21 M)
Resultado visual	Manchas de color	Caras reconocibles

Conclusiones

La lección práctica

La difusión enseña dos cosas. Primero, una idea poderosa: **partir un problema imposible en muchos pasos fáciles** —quitar ruido poco a poco— es lo que ha llevado a la IA generativa a su nivel actual. Y segundo, una lección de oro para cualquiera que trabaje con sistemas complejos: cuando algo falla, **el culpable no siempre es lo que entrenas**. Aquí el modelo era impecable y el error estaba en cómo se le pedía trabajar; un buen diagnóstico ahorró tirar a la basura un modelo que estaba perfecto. Saber *dónde* mirar vale tanto como saber construir.

PARA TERMINAR

Los cuatro, cara a cara

No existe «el mejor»: cada uno hace un trato distinto

Tras este recorrido, la conclusión más importante es que **ninguno de los cuatro modelos es «el mejor»**. Cada uno renuncia a algo para ganar otra cosa: el VAE cede algo de nitidez a cambio de poder inventar de forma ordenada; la GAN gana nitidez a cambio de un entrenamiento delicado; la difusión logra la mejor calidad a cambio de ser más lenta al generar. Elegir un modelo es, en realidad, **elegir qué compromiso te conviene**.



Tabla comparativa

PROPIEDAD	Autoencoder	VAE	GAN	Diffusion
¿Tiene encoder?	Sí	Sí	No	No
¿Genera caras nuevas?	No (reconstruye)	Sí (del prior)	Sí	Sí
Espacio latente	Determinista	Probabilístico $N(0, I)$	Ruido z	Ruido + tiempo t
Entrenamiento	Estable	Estable	Inestable (adversarial)	Estable
Muestreo	1 paso	1 paso	1 paso	Iterativo (lento)
Nitidez típica	Borrosa	Borrosa/suave	Nítida (con artefactos)	Buena

Galería comparada

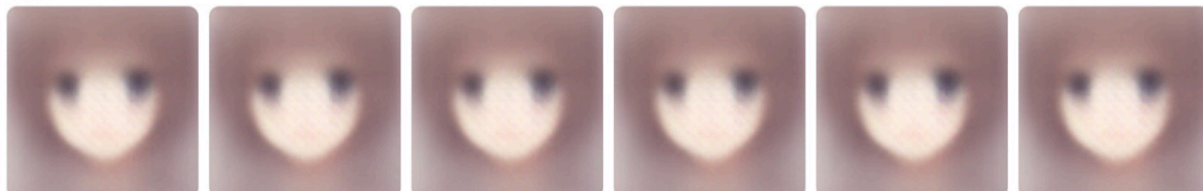
seed 42

Generar en todos

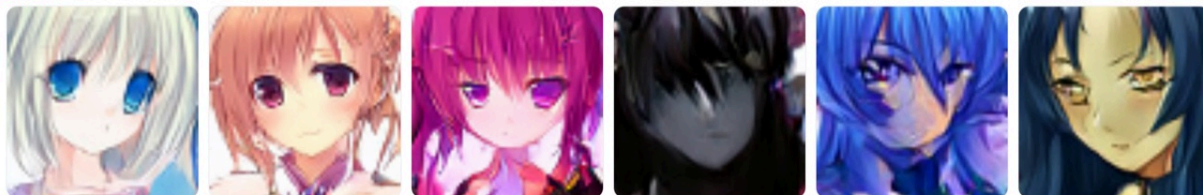
Autoencoder Reconstrucción - no genera caras nuevas



VAE Generadas - muestreo del prior

GAN Generadas - desde ruido z 

Diffusion Generadas - denoising



La pestaña Comparador: las salidas de los cuatro modelos puestas una al lado de otra, con la misma semilla de azar, para comparar de un vistazo su estilo y su calidad.

Pregunta sencilla	Autoencoder	VAE	GAN	Diffusion
¿Sabe «leer» una imagen (resumirla)?	Sí	Sí	No	No
¿Inventa caras nuevas?	No	Sí	Sí	Sí
¿Es fácil de entrenar?	Sí	Sí	Delicado	Sí
¿Genera rápido?	Al instante	Al instante	Al instante	Lento (muchos pasos)
¿Qué nitidez tiene?	Borrosa	Borrosa	Nítida	Buena

Las dos ideas que conviene llevarse

1. La generación de imágenes es una escalera de ideas, no una caja negra. Empieza por algo tan simple como «resume y reconstruye» (autoencoder) y, peldaño a peldaño —ordenar la memoria, poner a competir dos redes, quitar ruido paso a paso— se llega a la tecnología que hoy crea imágenes a partir de una frase. Entendido así, deja de parecer magia.

2. Mejorar es tan importante como construir. En cada modelo, el **bucle de realimentación con agentes de IA** —proponer, entrenar, medir y quedarse con lo que de verdad mejora— fue lo que convirtió un primer intento mediocre en algo que funciona. Y dejó lecciones que valen mucho más allá de la IA: que **más potencia no arregla un mal diseño** (el salto del autoencoder a la U-Net), y que **cuando algo falla, el culpable no siempre es lo obvio** (el modelo de difusión estaba perfecto; el fallo estaba en otra parte).

En una frase

Cuatro formas de enseñar a una máquina a crear imágenes, de la más simple a la más avanzada, contadas para que se entiendan; y un método para mejorarlas que, al final, es tan valioso como los propios modelos: **medir, comparar y quedarse con lo que funciona.**

Todo este laboratorio se ejecuta **en local**, en un portátil normal, sin enviar nada a la nube. El código y la guía para reproducirlo están disponibles de forma abierta (licencia MIT).

PONLO EN MARCHA

Descárgalo y Pruébalo

Todo corre en tu máquina — sin nube, sin cuentas, sin coste

El laboratorio es **100% local** y de código abierto. Está pensado para macOS con Apple Silicon —usa la GPU integrada vía MPS— pero también funciona en CPU. Necesitas **Python ≥ 3.11**, **Node ≥ 20** y **git**.

El repositorio

Todo el código está en GitHub: <https://github.com/rcanalescoder/generativelab.git>

1 · Clonar el proyecto

terminal

BASH

```
git clone https://github.com/rcanalescoder/generativelab.git
cd generativelab
```

2 · Arrancar el backend (FastAPI + PyTorch)

Crea un entorno de Python aislado, instala las dependencias y deja el servidor escuchando en el puerto 8000.

terminal

BASH

```
cd backend
python3 -m venv .venv           # entorno de Python aislado
source .venv/bin/activate
pip install -r requirements.txt  # FastAPI, PyTorch, NumPy...

python -m app.data.dataset      # descarga y cachea las 43.102 caras (solo la 1ª vez)
PYTORCH_ENABLE_MPS_FALLBACK=1 uvicorn app.main:app --port 8000
```

3 · Arrancar el frontend (React + Vite)

En otra terminal, instala y lanza la interfaz. Vite hace de «proxy» de /api hacia el backend.

terminal

BASH

```
cd frontend
npm install
npm run dev           # abre http://localhost:5173
```

Atajo: desde la raíz del proyecto, `./arrancar.sh` levanta (o reinicia) backend y frontend de una vez. Cuando ambos están vivos, la pastilla «**MPS activo**» del header se pone verde.

4 · (Opcional) Generar los modelos demo

Así cada pestaña aparece lista al instante con un modelo ya entrenado. Si te lo saltas, puedes entrenar desde la propia interfaz en modo «Rápido» en segundos.

terminal

BASH

```
# desde backend/ con el venv activado
python -m app.services.ae_service      # Autoencoder
python -m app.services.vae_service     # VAE
python -m app.services.gan_service     # GAN
python -m app.services.diffusion_service # Diffusion
```

Cómo está organizado el código

Dónde vive cada cosa — y dónde mirar para entender un modelo

El proyecto es un *monorepo* con dos mitades: un **frontend** (la interfaz que ves) y un **backend** (donde viven las redes neuronales y el entrenamiento). La regla mental para orientarse es sencilla:

la red	de cada modelo está en <code>backend/app/models/<modelo>.py</code> — ahí se definen el encoder, el decoder y todas las capas.
el entrenamiento	(y el muestreo, la interpolación...) está en <code>backend/app/services/<modelo>_service.py</code> .
la API	que une ambos mundos vive en <code>backend/app/routers/</code> : un endpoint por modelo.
la pantalla	de cada modelo es <code>frontend/src/tabs/<Modelo>Tab.tsx</code> ; la teoría de los botones «i», en <code>content/*.json</code> .

```

generativelab/
├── backend/
│   └── app/
│       ├── main.py           # FastAPI: monta los routers y carga los checkpoints
│       ├── device.py        # detecta el dispositivo (MPS / CPU)
│       ├── data/            # descarga y cache del dataset
│       ├── models/          # ← LAS REDES: ae · vae · gan · diffusion
│       ├── services/        # ← ENTRENAMIENTO + lógica (SSE, muestreo, interpolación)
│       ├── routers/         # un endpoint por modelo
│       └── checkpoints/     # pesos demo (*_demo.pt)
└── frontend/src/
    ├── components/ui/       # sistema de componentes propio (botones, sliders...)
    ├── components/lab/     # LatentMap, SampleGrid, TrainingChart, Filmstrip...
    ├── content/            # teoría de los botones «i» (JSON, no código)
    ├── lib/                # cliente HTTP/SSE (uno por modelo)
    └── tabs/               # una pantalla por modelo
  
```

Un patrón se repite en los cuatro modelos: el entrenamiento corre en un **hilo aparte** que va emitiendo eventos por **SSE** (por eso la curva de pérdida avanza en vivo). Y los **checkpoints demo son intocables**: entrenar desde la app solo cambia el modelo en memoria; nunca pisa el fichero base.

El código clave, modelo a modelo

Para cada uno: dónde se define la red, dónde se entrena y algún truco interesante

Autoencoder

Un encoder que comprime y un decoder simétrico que reconstruye, unidos por el «cuello de botella».

backend/app/models/ae.py

LA RED

```
class ConvAutoencoder(nn.Module):
    def __init__(self, latent_dim=128):
        c1, c2, c3, c4 = 32, 64, 128, 256
        # encoder: 4 convoluciones, cada una parte el lado por la mitad
        self.encoder = nn.Sequential(
            _down(3, c1, norm=False), # 64 → 32
            _down(c1, c2), # 32 → 16
            _down(c2, c3), # 16 → 8
            _down(c3, c4) # 8 → 4
        )
        self.fc_enc = nn.Linear(c4*4*4, latent_dim) # 4096 → z (128): el cuello
        # decoder: el espejo del encoder, de z de vuelta a 64×64×3
        self.fc_dec = nn.Linear(latent_dim, c4*4*4)
        self.decoder = nn.Sequential(
            _up(c4, c3), _up(c3, c2), _up(c2, c1), # 4 → 8 → 16 → 32
            nn.ConvTranspose2d(c1, 3, 4, 2, 1), nn.Sigmoid() # 32 → 64
```

La red. El truco está en `fc_enc`: 4096 números se estrujan a solo 128 — el cuello de botella que obliga a la red a quedarse con lo esencial.

backend/app/services/ae_service.py

ENTRENAMIENTO

```
opt = torch.optim.Adam(model.parameters(), lr=hp.learning_rate)
crit = nn.L1Loss() if hp.loss == "l1" else nn.MSELoss()
for epoch in range(1, epochs + 1):
    for batch in self._iter_batches(indices, hp.batch_size):
        recon, _ = model(batch) # imagen → z → reconstrucción
        loss = crit(recon, batch) # ¿se parece a la original?
        loss.backward(); opt.step() # corrige los pesos
    yield {"type": "epoch", "loss": epoch_loss} # evento SSE → curva en vivo
```

El bucle de entrenamiento. No hay etiquetas: el «profesor» es la propia imagen de entrada.

backend/app/services/ae_service.py

EXTRA · INTERPOLACIÓN

```
# Caminar en línea recta entre dos caras A y B dentro del espacio latente
za, skips_a = model.encode_with_skips(xa) # código completo de A
zb, skips_b = model.encode_with_skips(xb) # código completo de B
for s in range(steps):
    alpha = s / (steps - 1) # de 0 (A) a 1 (B)
    z = (1 - alpha) * za + alpha * zb # mezcla de los dos códigos
    skips = [(1-alpha)*a + alpha*b for a, b in zip(skips_a, skips_b)]
    img = model.decode(z, skips)[0] # la cara intermedia
```

La interpolación A→B: si las caras intermedias salen suaves y con sentido, el modelo aprendió un espacio «con estructura», no un simple diccionario.

VAE

El mismo esqueleto, pero el encoder produce una distribución y se entrena con un término extra que ordena la memoria.

backend/app/models/vae.py

LA RED · EL TRUCO CLAVE

```
def encode(self, x):                #  $x \rightarrow (\mu, \log\sigma^2)$ : una DISTRIBUCIÓN, no un punto
    h = self.encoder(x).flatten(1)
    return self.fc_mu(h), self.fc_logvar(h)

def reparameterize(self, mu, logvar): # el «truco de reparametrización»
    std = torch.exp(0.5 * logvar)
    eps = torch.randn_like(std)      # ruido aleatorio  $\epsilon \sim N(0, I)$ 
    return mu + eps * std            #  $z = \mu + \sigma \cdot \epsilon$  (derivable: deja entrenar)

def sample(self, n, device):         # GENERAR caras nuevas: muestrear del prior
    z = torch.randn(n, self.latent_dim, device=device)
    return self.decode(z)
```

La clave del VAE es reparameterize: muestrear parece incompatible con entrenar (el azar no tiene derivada), pero separando el ruido ϵ sí se puede. Y como el latente queda ordenado, sample le inventa caras desde cero.

backend/app/services/vae_service.py

ENTRENAMIENTO

```
recon, mu, logvar, _ = model(batch)
recon_loss = recon_crit(recon, batch) # 1) que se parezca al original
kl_loss = _kl_divergence(mu, logvar)  # 2) que el latente sea ordenado,  $\sim N(0, I)$ 
loss = recon_loss + hp.beta * kl_loss #  $\beta$  decide el equilibrio entre ambos
loss.backward(); opt.step()
```

La pérdida tiene dos partes en tensión: reconstruir bien vs. mantener el «mapa de memoria» ordenado para poder generar. El parámetro β decide a cuál dar más peso.

GAN

Dos redes que compiten: un generador que falsifica y un discriminador que detecta falsificaciones.

backend/app/models/gan.py

LAS DOS REDES

```
class Generator(nn.Module):          # ruido  $z \rightarrow$  imagen 64×64 (no hay encoder)
    def __init__(self, z_dim=100):
        self.net = nn.Sequential(
            nn.ConvTranspose2d(z_dim, 512, 4, 1, 0), ..., # 1 → 4 → ... → 64
            nn.Tanh())                                     # salida en [-1, 1]

class Discriminator(nn.Module):      # imagen → un número (¿real o falsa?)
    def __init__(self):
        self.net = nn.Sequential(
            _d_down(3, 64), ..., nn.Conv2d(512, 1, 4, 1, 0)) # 64×64 → 1 logit
```

El generador (de ruido a cara) y el discriminador (de cara a veredicto): simétricos e inversos.

```
# 1) Entrenar al DISCRIMINADOR: que acierte real=1, falsa=0
fake = generator(z)
out_real = discriminator(real)
out_fake = discriminator(fake.detach()) # detach: aún no toca al generador
loss_d = criterion(out_real, real_labels) + criterion(out_fake, fake_labels)
loss_d.backward(); opt_d.step()

# 2) Entrenar al GENERADOR: que el juez trague sus falsas como reales
loss_g = criterion(discriminator(fake), real_labels) # etiqueta = «real»
loss_g.backward(); opt_g.step()
```

El tira y afloja: el discriminador mejora detectando y el generador mejora engañando. De esa competición salen caras cada vez más nítidas... mientras el equilibrio no se rompa (por eso es inestable).

Diffusion

La red aprende a quitar ruido. Entrenar es ensuciar una imagen y pedirle que adivine el ruido añadido.

```
def q_sample(sched, x0, t, noise): # «ensuciar» la imagen x0 hasta el paso t
    #  $x_t = \sqrt{\alpha_t} \cdot x_0 + \sqrt{(1-\alpha_t)} \cdot \epsilon$  (una mezcla de imagen y ruido)
    sqrt_acp = gather(sched.sqrt_alphas_cumprod, t)
    sqrt_om = gather(sched.sqrt_one_minus_alphas_cumprod, t)
    return sqrt_acp * x0 + sqrt_om * noise
```

El proceso «forward» añade ruido de forma controlada: cuanto mayor es t , más ruido, hasta que solo queda «nieve».

```
t = torch.randint(0, timesteps, (b,)) # un nivel de ruido al azar por imagen
noise = torch.randn_like(batch) # el ruido que vamos a añadir
x_t = q_sample(sched, batch, t, noise) # imagen ensuciada
eps_pred = model(x_t, t) # la red ADIVINA el ruido
loss = F.mse_loss(eps_pred, noise) # ¿acertó el ruido?
loss.backward(); opt.step(); ema.update(model)
```

Entrenar es sorprendentemente simple: ensucia, predice el ruido, corrige. (La EMA guarda una media suave de los pesos, que da mejores muestras al generar.)

```
x = torch.randn(n, 3, img_size, img_size) # partir de ruido PURO
for t in pasos: # de mucho ruido a nada
    eps = model(x, t) # ruido estimado en este paso
    x0 = (x - sqrt(1- $\bar{\alpha}_t$ )*eps) / sqrt( $\bar{\alpha}_t$ ) # estima la cara limpia...
    x = sqrt( $\bar{\alpha}_{prev}$ )*x0 + sqrt(1- $\bar{\alpha}_{prev}$ )*eps # ...y retrocede un poco el ruido
```

El muestreo «reverse» con DDIM: partiendo de pura «nieve», repite quitar-un-poco-de-ruido decenas de veces hasta que emerge una cara. Es exactamente lo que ves en el panel «Proceso de difusión».

CRÉDITOS

Licencia y autoría

Código abierto bajo licencia MIT

Autor	Roberto Canales Mora — con Claude Chat / Code (Anthropic) como par de programación.
Licencia	MIT (texto completo abajo): libertad para usar, copiar, modificar y distribuir el código.
Repositorio	https://github.com/rcanalescoder/generativelab.git
Dataset	huggan/anime-faces (CC0): 43.102 caras de 64x64 píxeles.

■ Licencia MIT

LICENSE

MIT

Licencia MIT

Copyright (c) 2026 Roberto Canales Mora

Por la presente se concede permiso, sin cargo, a cualquier persona que obtenga una copia de este software y de los archivos de documentación asociados (el "Software"), para utilizar el Software sin restricción, incluyendo sin limitación los derechos a usar, copiar, modificar, fusionar, publicar, distribuir, sublicenciar y/o vender copias del Software, y a permitir a las personas a las que se les proporcione el Software a hacer lo mismo, sujeto a las siguientes condiciones:

El aviso de copyright anterior y este aviso de permiso se incluirán en todas las copias o partes sustanciales del Software.

EL SOFTWARE SE PROPORCIONA "TAL CUAL", SIN GARANTÍA DE NINGÚN TIPO, EXPRESA O IMPLÍCITA, INCLUYENDO PERO NO LIMITADO A GARANTÍAS DE COMERCIALIZACIÓN, IDONEIDAD PARA UN PROPÓSITO PARTICULAR Y NO INFRACCIÓN. EN NINGÚN CASO LOS AUTORES O PROPIETARIOS DE LOS DERECHOS DE AUTOR SERÁN RESPONSABLES DE NINGUNA RECLAMACIÓN, DAÑOS U OTRAS RESPONSABILIDADES, YA SEA EN UNA ACCIÓN DE CONTRATO, AGRAVIO O DE OTRO MODO, QUE SURJA DE, FUERA DE O EN CONEXIÓN CON EL SOFTWARE O EL USO U OTROS NEGOCIOS EN EL SOFTWARE.

Hecho para aprender, compartir y experimentar. ✦